

GraphOne / FrontierAtlas Intelligence Ingestion Architecture

Technical Design Specification: Scaling from 3-Day Trial to 500,000+ Multi-Dimensional Entities

1. System Paradigm: Trial Implementation vs. 500k+ Production Architecture

To satisfy the rigorous requirements of the assessment, we explicitly bifurcate the operational footprint between the **3-Day Local Trial** and the **Distributed Production Architecture**. The local trial operates under a hard storage budget of **< 1 GB (target < 700 MB)**, leveraging an in-memory streaming ingestion pipeline, zero raw HTML persistence, zero PDF storage, bounded SQLite indexing, and direct API adapters. Conversely, the production system is designed to acquire and continuously reconcile **500,000+ entities (lakhs of records)** across startups, products, research papers, jobs, and news without manual intervention.

Component	3-Day Trial Implementation	500k+ Production Target Architecture
Concurrency	asyncio + aiohttp Semaphore (15-20 conn)	Distributed Ray / Celery Workers on Kubernetes (HPA)
Message Queue	In-memory async Queue & Generator Streams	Apache Kafka (Partitioned Event Streams) + Redis
Storage Layer	WAL-compacted SQLite (< 5 MB footprint)	Sharded PostgreSQL 16 + Citus + MinIO S3 Lakehouse
Intelligence Graph	Relational Foreign Keys + SQLite Indices	Neo4j / Amazon Neptune + pgvector (Embedding Index)
LLM Processing	Gemini Flash -> Groq -> DeepSeek Fallback	vLLM Local Self-Hosted Mesh + Cloud Fallback Gateway

2. Scaling to 500,000+ Records Without Bottlenecks

Scaling data acquisition by orders of magnitude (from 1,000 to 500,000+ records) requires eliminating shared locks, decoupling discovery from extraction, and enforcing horizontal backpressure across all ingestion nodes:

- **Distributed Frontier Coordinator:** A centralized Source Registry schedules periodic discovery jobs across millions of seed URLs. URL states are tracked in a partitioned Redis Bloom Filter (100M keys ~12MB RAM) combined with Kafka topic partitions keyed by domain hashes.
- **Stateless Async Scraping Workers:** Crawler pods run asynchronously using aiohttp for high-throughput HTTP/JSON endpoints and headless Chromium instances (Playwright Cluster) for dynamic DOM rendering. Workers scale dynamically via KEDA based on Kafka queue lag.
- **Zero-Accumulation Streaming Buffer:** Scraped HTML/text is streamed directly into an ephemeral pipeline. Bounded text segments are emitted to extraction queues; raw markup is dumped into S3/MinIO cold archival tier (lifecycle rule: 7 days) and immediately purged from worker memory.
- **Multi-Stage Parallel Ingestion:** Vertical-specific ingestion pipelines (Startups, Products, Papers, News, Jobs) execute independently on dedicated worker pools, guaranteeing that high-volume arXiv paper streams never starve real-time news monitors.

3. Resilient LLM Extraction Engine: Managing 413s & 429s

Large-scale LLM extraction fails when pipelines naively blast raw DOM trees into model endpoints. Our architecture employs a multi-tiered defense mechanism that simultaneously eliminates context overflows (HTTP 413) and rate limits (HTTP 429).

A. Context Window Governance & HTTP 413 Payload Reduction

Web pages contain 70–90% noise (inline JavaScript, SVG assets, CSS style blocks, tracking pixels, cookie consent popups, and boilerplate footers). Before any content is forwarded to an LLM, it traverses the **Intelligent Content Reduction Pipeline**:

- Deterministic DOM Stripping:** Removes all script, style, nav, footer, header, and cookie tags via parser-level AST decomposition.
- Whitespace Normalization & Density Extraction:** Collapses multiple blank lines and extracts only semantically dense article and metadata nodes.
- Token Budget Estimator:** Calculates token footprint (approx. 4 characters/token). If content exceeds MAX_INPUT_CHARS (12,000 chars ~ 3,000 tokens), it invokes semantic chunking across natural paragraph and sentence boundaries.
- Dynamic 413 Catch & Halve Strategy:** If an API returns HTTP 413 or payload-too-large, the orchestrator catches the exception, halves the payload window (reduction_factor=0.5), and retries with the compacted payload before escalating to secondary providers.

B. Rate Limiting (HTTP 429), Backoff & Multi-Tier Provider Fallback

Commercial LLM providers enforce tight Requests-Per-Minute (RPM) and Tokens-Per-Minute (TPM) quotas. The pipeline deploys an adaptive fallback chain:

Step 1: Primary Gateway	Step 2: Secondary Failover	Step 3: Tertiary Failover	Step 4: Deterministic Fallback
Gemini Flash High-throughput, low latency structured extraction with JSON mode.	Groq (Llama 3.1) Triggered on Gemini 429, timeout, or service disruption.	DeepSeek Chat Triggered if Groq exhausts retry budget or encounters 429.	Rule Extractor Zero-hallucination heuristic parse to preserve pipeline continuity.

- Exponential Backoff with Full Jitter:** Retries follow $t = \min(t_{max}, t_{base} * 2^{attempt}) * uniform(0.5, 1.5)$. This desynchronizes thundering herd requests across concurrent scraping tasks.
- Respect for Retry-After Headers:** When upstream providers specify a mandatory cooling-off period, the worker suspends execution for the exact duration before re-engaging.

C. Zero-Tolerance Hallucination Prevention

Hallucinated records disqualify venture intelligence graphs. To guarantee 100% data fidelity:

- Strict Schema Contracts:** All extractions must validate against Pydantic models with constrained enums (e.g. FREE, FREEMIUM, PAID, ENTERPRISE).
- Mandatory Source Provenance:** Every record retains its immutable source.url and collectedAt timestamp.
- Null-on-Missing:** Missing data (e.g. employee count, pricing tier) is set strictly to null; speculative generation is forbidden.
- Bounded Evidence Snippets:** For unstructured text, an auditing snippet of <= 500 characters is preserved to verify provenance without disk bloating.

4. Freshness Tracking & Distributed Idempotency

News and jobs require rigorous freshness guarantees (≤ 24 hours). The pipeline implements a **Deterministic Extraction Hierarchy**:

- RSS/Atom Protocol Headers:** pubDate, published, updated tags parsed via RFC-2822/ISO-8601.
- Microdata & JSON-LD:** datePublished, uploadDate in schema.org structured scripts.
- OpenGraph Metadata:** og:published_time and article:published_time tags.
- Semantic DOM Tags:** HTML5 <time datetime='...'> attributes.
- Deterministic Relative Parsing:** Expressions like '2 hours ago' or 'yesterday' computed against UTC reference time.

Freshness Filter: Any article or job published earlier than now - 24 hours is strictly rejected. A 15-minute future tolerance is enforced to accommodate server clock drift.

Distributed Idempotency: Each record computes a composite fingerprint: SHA256(canonical_url + published_timestamp). In production, a distributed Redis set and PostgreSQL ON CONFLICT DO NOTHING clause prevent duplicate ingestion across worker nodes.

5. Deterministic Entity Resolution Engine

Raw venture data is notoriously noisy (e.g. 'OpenAI, Inc.', 'Open AI', 'openai-org' all represent the same entity). The resolution pipeline operates as follows:

- Normalization Pipeline:** Unicode NFKD normalization -> lowercase conversion -> corporate suffix stripping (Inc, LLC, Corp, Ltd, PBC, GmbH, AG, SAS, Labs, Tech) -> punctuation strip -> whitespace collapse.
- Exact Alias Matching:** Matches against a curated canonical dictionary of 50+ prominent AI companies and products (100% confidence).
- Conservative RapidFuzz Scoring:** Employs fuzz.token_sort_ratio. Scores $\geq 88.0\%$ automatically canonicalize; scores between 75–87% are flagged for review; scores $< 75\%$ retain distinct identity to avoid false merges.
- Auditable Entity Mapping Log:** Every resolution records raw_name, canonical_name, method, confidence, and source_url.

6. Storage Strategy: Primary Database, Vector & Graph Storage

To map complex venture relationships across founders, companies, products, patents, and hiring trends at 500k+ scale:

Storage Layer	Technology	Architectural Justification & Role
Primary Relational DB	PostgreSQL 16 + Citus	ACID compliance, relational integrity, row-level idempotency via unique hash constraints, and horizontal sharding across millions of records.
Vector Semantic Index	pgvector / Qdrant	Enables high-speed cosine similarity search across research paper abstracts, product descriptions, and news embeddings for semantic deduplication.
Intelligence Graph DB	Neo4j / Amazon Neptune	Enables multi-hop graph traversals: (Founder)-[:FOUNDED]->(Startup)-[:PRODUCES]->(Product)-[:IMPLEMENTS]->(ResearchPaper).

Local Trial Compliance Note: In our local implementation, SQLite in WAL mode delivers the exact normalized schema and unique constraints of PostgreSQL while consuming only ~3.0 MB of disk space, easily satisfying the < 1 GB hard storage budget.